

# TKStorageBack

Backend учёта клубного снаряжения  
Техническая документация и руководство по передаче

---

Kotlin · Spring Boot 3.5 · MongoDB · fly.io · Google Apps Script

Состояние кода на 18.09.2026

Прод: [tkstorage-api.fly.dev](https://tkstorage-api.fly.dev)

# Содержание

|                                                         |           |
|---------------------------------------------------------|-----------|
| <b>1. Что это за система</b>                            | <b>3</b>  |
| 1.1. Из чего состоит контур                             | 3         |
| 1.2. стек и требования                                  | 3         |
| <b>2. Быстрый старт для нового разработчика</b>         | <b>3</b>  |
| <b>3. Модель данных</b>                                 | <b>4</b>  |
| 3.1. equip_items — карточки снаряжения                  | 4         |
| 3.2. club_users — справочник людей                      | 4         |
| 3.3. history — журнал изменений                         | 5         |
| <b>4. Доменные правила</b>                              | <b>5</b>  |
| 4.1. Бизнес-ключ человека                               | 5         |
| 4.2. Нормализация (util/Normalize.kt)                   | 5         |
| 4.3. Поиск                                              | 6         |
| 4.4. Привязки: как они ставятся и теряются              | 6         |
| 4.5. Служебные места                                    | 6         |
| <b>5. HTTP API</b>                                      | <b>6</b>  |
| 5.1. Общие правила                                      | 6         |
| 5.2. Предметы                                           | 7         |
| 5.3. Люди                                               | 7         |
| <b>6. Синхронизация с гугл-таблицами</b>                | <b>7</b>  |
| 6.1. Ночной прогон                                      | 7         |
| 6.2. Импорт справочника людей                           | 8         |
| 6.3. Импорт и выгрузка снаряжения                       | 8         |
| 6.4. Устойчивость к сбоям Google                        | 8         |
| <b>7. Развёртывание и эксплуатация</b>                  | <b>9</b>  |
| 7.1. Прод                                               | 9         |
| 7.2. Типовые операции                                   | 9         |
| <b>8. Известные проблемы и технический долг</b>         | <b>9</b>  |
| 8.1. Хрупкий ключ: правка анкеты молча снимает привязки | 9         |
| 8.2. Наблюдаемость                                      | 10        |
| 8.3. Безопасность                                       | 10        |
| 8.4. Данные и совместимость                             | 10        |
| 8.5. Тесты                                              | 10        |
| <b>9. Хронология значимых изменений</b>                 | <b>11</b> |
| <b>10. Глоссарий</b>                                    | <b>11</b> |

## 1. Что это за система

TKStorageBack — backend учёта клубного снаряжения. Он хранит карточки предметов, справочник людей и историю всех изменений, а также синхронизируется с гугл-таблицами, в которых клуб вёл учёт до появления сервиса и продолжает вести отчётность сейчас.

Основной потребитель — мобильное приложение кладовщика. Через него выдают и принимают снаряжение, ищут предметы и людей, смотрят, что у кого на руках. Второй потребитель — сами гугл-таблицы: ночью сервис забирает из них справочник людей и выгружает обратно актуальное состояние снаряжения.

### 1.1. Из чего состоит контур

```
[Мобильное приложение]
  | HTTPS, заголовок X-App-Version-Code
  v
[Spring Boot API] ---> [MongoDB]
tkstorage-api         equip_items / club_users / history
fly.io, регион fra
  |
  | CSV через Google Apps Script (три отдельных развёртывания)
  v
[Google Sheets] <--- [Яндекс.Форма] (анкеты новых членов клуба)
снаряжение
анкеты людей
```

Три точки интеграции с Google живут в application.properties как три URL Apps Script: выгрузка снаряжения в таблицу (export), импорт снаряжения из таблицы (import) и импорт справочника людей (users). Исходники этих скриптов в репозитории HE лежат — они редактируются в интерфейсе Google Apps Script. Единственная сохранённая копия, example.txt в корне, относится к прежней архитектуре, когда приложение ходило в таблицу напрямую, без backend. Держите это в голове: скрипты — часть системы, но вне контроля версий.

### 1.2. Стек и требования

| Компонент      | Версия / значение                                                    |
|----------------|----------------------------------------------------------------------|
| Язык           | Kotlin 1.9.25 (JVM toolchain 17)                                     |
| Фреймворк      | Spring Boot 3.5.11 (web, data-mongodb)                               |
| Хранилище      | MongoDB (авто-создание индексов включено)                            |
| Сборка         | Gradle (wrapper в репозитории), задача bootJar                       |
| Excel-выгрузка | Apache POI 5.3.0 (xssf)                                              |
| Хостинг        | fly.io, приложение tkstorage-api, регион fra                         |
| Образ          | gradle:8.14-jdk17 (сборка) -> eclipse-temurin:17-jre-alpine (запуск) |

## 2. Быстрый старт для нового разработчика

```
git clone <репозиторий> && cd TKStorageBack
# нужен локальный MongoDB на 27017 (база equipdb)
./gradlew test           # юнит-тесты
./gradlew bootRun --args="--appscript.sync.cron=-"  # см. предупреждение ниже
curl -s localhost:8080/health
curl -s -H "X-App-Version-Code: 4" --get \
    --data-urlencode "query=иванов" localhost:8080/users/search
```

**ВАЖНО** при локальном запуске. В application.properties прописаны боевые URL Apps Script и включён ночной крон (04:30 МСК). Локальный инстанс, доживший до этого времени, выгрузит свою тестовую базу снаряжения поверх реальной таблицы клуба. Запускайте локально только с appscript.sync.cron=- — значение "-" отключает планировщик.

Импорт справочника (POST /users/import/sheets) на локальном инстансе безопасен: он только читает гугл-таблицу и пишет в локальную Mongo. Опасна именно выгрузка, потому что она перезаписывает таблицу целиком.

## 3. Модель данных

Три коллекции MongoDB. У предметов и людей есть собственный бизнес-идентификатор id (уникальный индекс), отдельный от mongoid — именно он ездит в API и в гугл-таблицы.

### 3.1. equip\_items — карточки снаряжения

| Поле                         | Смысл                                                                |
|------------------------------|----------------------------------------------------------------------|
| id                           | Инвентарный номер, уникален, числовой строкой ("3044")               |
| category, brand, name, color | Категория, производитель, название, цвет                             |
| weigh, quality               | Вес и состояние — строки, произвольный текст                         |
| location                     | Где предмет: ФИО человека, "Склад", "Неизвестно" или свободный текст |
| locationUserId               | Привязка к записи справочника. Пусто = привязки нет                  |
| event, info, date            | Мероприятие, заметка, дата последнего движения                       |

Пара location + locationUserId — центральная идея модели. Текст показывается человеку, привязка связывает предмет с записью справочника. Сервис следит, чтобы они не разъезжались: у привязанного предмета текст всегда равен каноническому ФИО из справочника.

### 3.2. club\_users — справочник людей

| Поле                            | Смысл                                                    |
|---------------------------------|----------------------------------------------------------|
| id                              | Бизнес-ключ: хэш от ФИО и даты рождения (см. раздел 4.1) |
| lastName, firstName, middleName | ФИО по отдельности, как в анкете                         |
| fullName                        | Собранное ФИО — то, что видит пользователь               |
| birthDate / birthDateNormalized | Дата рождения как в таблице / приведённая к дд.ММ.гггг   |
| telegram, phone, email, vk      | Контакты                                                 |

| Поле                                                        | Смысл                                                              |
|-------------------------------------------------------------|--------------------------------------------------------------------|
| intake, joinYear, tourTraining, rank, rankExtra, experience | Набор, год вступления, подготовка, разряды, опыт                   |
| occupation, address, comment                                | Служебные поля анкеты, наружу не отдаются                          |
| searchText                                                  | Предвычисленная строка для поиска: ФИО + телеграм, нормализованные |
| system                                                      | true у служебных записей "Склад" и "Неизвестно"                    |
| updatedAt                                                   | Время импорта, дд.ММ.гггг ЧЧ:мм                                    |

Справочник целиком перезаписывается при каждом импорте: deleteAll, затем saveAll. Редактировать людей через API нельзя — единственный источник правды это гугл-таблица анкет.

### 3.3. history — журнал изменений

Каждое изменение предмета через API пишет запись: действие, ключевые поля и кто это сделал (keyholderName приходит от клиента). Изменившиеся поля записываются в формате "было-->стало", неизменившиеся — как есть; при отдаче наружу неизменившиеся поля обнуляются, чтобы в приложении была видна только разница.

Чего в истории нет. Снятие привязок при импорте справочника пишется в базу напрямую, минуя журнал. То есть предмет может "сам" оказаться в разборе, и в его истории не будет никаких следов. Это известный пробел, см. раздел 8.2.

## 4. Доменные правила

### 4.1. Бизнес-ключ человека

У анкет из гугл-формы нет собственного идентификатора, поэтому ключ вычисляется из данных: первые 16 hex-символов SHA-256 от нормализованных фамилии, имени, отчества и даты рождения.

```
ClubUser.buildId(lastName, firstName, middleName, birthDate) =
    sha256( "фамилия|имя|отчество|дд.ММ.гггг" ).hex.take(16)
```

Следствие, которое нужно понимать до того, как что-то менять: ключ зависит от содержимого анкеты. Правка ФИО или даты рождения в таблице — это новый id и исчезновение старого. Все привязки предметов к исчезнувшему id снимаются при ближайшем импорте (раздел 8.1).

### 4.2. Нормализация (util/Normalize.kt)

- text(s) — trim, нижний регистр, ё заменяется на е, схлопывание пробелов. Основа сравнения и поиска.
- birthDate(raw) — приводит дату к дд.ММ.гггг по явному списку форматов; что не разобралось, возвращается как нормализованный текст.
- digits(s) — оставляет только цифры.
- sha256Short(s) — 16 hex-символов от SHA-256.

Список форматов дат сознательно закрытый, а не "умный парсер": 01-02-2004 — это 1 февраля по-русски и 2 января по-американски, и молчаливая смена трактовки означала бы смену ключей у части людей. Поддерживаются точки, дефисы и слэши, двузначный год с базой 1930 (иначе 25.07.97 стало бы

2097 годом) и ISO-формат 2004-01-24, в том числе с хвостом времени.

ISO-ветка появилась 18.09.2026: подключённая неделей раньше выгрузка анкет из Яндекс.Формы пишет даты в таблицу с дефисами. Люди, заполнившие анкету и до, и после интеграции, получали два разных ключа и двоились в поиске. Фикс схлопнул 20 таких пар.

### 4.3. Поиск

Поиск людей идёт по одному предвычисленному полю `searchText` регулярным выражением без учёта регистра, все слова запроса должны найтись. Запрос короче двух символов отклоняется, ведущая собака в телеграмме снимается. Отдаётся не больше 50 совпадений, при переполнении выставляется `hasMore` и просьба уточнить запрос. Служебные записи показываются первыми.

Поиск предметов устроен иначе: регулярка по шести полям карточки (`location`, `name`, `brand`, `category`, `color`, `event`), без предвычисленного поля и без лимита.

### 4.4. Привязки: как они ставятся и теряются

Логика в `EquipService.resolveLocation`, три ветки по значению `locationUserId` в запросе:

| Что прислал клиент      | Что делает сервис                                                                                                                                    |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Непустой id             | Ставит привязку, текст <code>location</code> берёт из справочника. Неизвестный id — отказ с сообщением "Пользователь не найден, обновите справочник" |
| Пустая строка           | Привязку снимает, текст остаётся свободным                                                                                                           |
| Поле не прислано (null) | Сохраняет старую привязку, только если текст местоположения не менялся (сравнение нормализованное). Изменили текст — привязка снимается              |

Практическое следствие третьей ветки: клиент, который не умеет привязки и просто пишет ФИО текстом, каждой выдачей создаёт предмет без привязки. Такие предметы попадают на экран разбора. Если разбор регулярно пополняется — проверьте версию приложения у тех, кто выдаёт снаряжение.

### 4.5. Служебные места

В справочнике всегда есть две системные записи с фиксированными id: `warehouse` ("Склад") и `unknown` ("Неизвестно"). Они дописываются при каждом импорте и не зависят от таблицы. При импорте снаряжения тексты "склад", "новый склад" и "неизвестно" автоматически превращаются в привязки к этим записям — это единственное место, где привязка ставится автоматически, и это не догадка, а явное соответствие. Привязку к человеку ставит только человек через приложение: сервис принципиально не сопоставляет ФИО по тексту.

## 5. HTTP API

### 5.1. Общие правила

- Каждый запрос, кроме `/health`, `/items/export` и `/items/export/csv`, обязан нести заголовок `X-App-Version-Code` с числом не меньше 4. Иначе 403 и сообщение "Пожалуйста обновите приложение!".
- Аутентификации нет вообще: заголовок версии — не защита, его подставит кто угодно.
- CORS открыт для всех `origin`, методы GET, POST, PUT.

- Ответы единообразны: поле success, полезная нагрузка и необязательное message. Пустые поля в JSON не сериализуются.
- Время везде GMT+3, формат дд.ММ.гггг ЧЧ:мм.

## 5.2. Предметы

| Метод и путь              | Назначение                                                    |
|---------------------------|---------------------------------------------------------------|
| GET /items                | Все карточки                                                  |
| GET /items/{id}           | Одна карточка                                                 |
| POST /items/add           | Добавить. Тело: newItem, keyholderName                        |
| PUT /items/update/{id}    | Изменить. Тело: newItem, keyholderName, historyAction         |
| GET /items/free-id        | Следующий свободный инвентарный номер                         |
| GET /items/unbound        | Предметы со свободным текстом вместо привязки — экран разбора |
| GET /items/search?query=  | Поиск по шести полям карточки                                 |
| GET /items/{id}/history   | История предмета                                              |
| GET /items/export/csv     | Вся база в CSV (без заголовка версии!)                        |
| GET /items/export/xls     | Вся база в xls                                                |
| POST /items/export/sheets | Выгрузить снаряжение в гугл-таблицу                           |
| POST /items/import/sheets | Импортировать снаряжение из гугл-таблицы                      |

## 5.3. Люди

| Метод и путь              | Назначение                                                   |
|---------------------------|--------------------------------------------------------------|
| GET /users/search?query=  | Поиск по ФИО и телеграму, до 50 результатов                  |
| GET /users/{id}           | Профиль. Адрес, род занятий и комментарий наружу не отдаются |
| GET /users/{id}/items     | Что у человека на руках                                      |
| POST /users/import/sheets | Импорт справочника из гугл-таблицы                           |

```
# типовой вызов
curl -s -H "X-App-Version-Code: 4" --get \
  --data-urlencode "query=ропбачева" \
  https://tkstorage-api.fly.dev/users/search

# ручной импорт справочника (и сверка привязок)
curl -s -X POST -H "X-App-Version-Code: 4" \
  https://tkstorage-api.fly.dev/users/import/sheets
```

# 6. Синхронизация с гугл-таблицами

## 6.1. Ночной прогон

Планировщик запускается по cron-выражению из appscript.sync.cron в зоне Europe/Moscow, сейчас это 04:30. Значение "-" отключает задачу целиком. Шага два и порядок важен: сначала импорт справочника, затем выгрузка снаряжения. Импорт может снять протухшие привязки и поправить тексты, и эти изменения тоже должны уехать в таблицу. Выгрузка выполняется даже если импорт не удался — дневные правки из приложения всё равно надо дослать.

Привязки живут только в Mongo, пока их не выгрузили: id попадают в таблицу и лишь оттуда переживают следующий импорт снаряжения.

## 6.2. Импорт справочника людей

Ожидается CSV в 17 колонок без заголовка: фамилия, имя, отчество, набор, телефон, email, телеграм, vk, род занятий, дата рождения, адрес, год вступления, турподготовка, разряд, дополнительный разряд, опыт, комментарий.

- Строка, где первая колонка нормализуется в "фамилия", пропускается — это случайно попавший заголовок.
- Строки шире 17 колонок пропускаются как испорченные, короче — добиваются пустыми значениями.
- Строка без фамилии и имени пропускается: ни ключа, ни поиска из неё не получится.
- Дубли по ключу схлопываются: берётся поздняя анкета, пустые поля добиваются из ранней.
- Предохранитель MIN\_KEEP\_RATIO: если в выгрузке меньше половины от текущего размера справочника, импорт отменяется целиком — так выглядит обрезанный ответ Apps Script под квотой.
- Семафор не даёт ручному и плановому импорту наложиться: два deleteAll внахлест оставили бы справочник наполовину пустым.
- После записи вызывается сверка привязок со свежим справочником.

Ответ сообщает, сколько строк пропущено, сколько дублей схлопнуто, сколько привязок снято и сколько текстов поправлено. Эти же цифры пишутся в лог при ночном прогоне.

## 6.3. Импорт и выгрузка снаряжения

Выгрузка формирует CSV из 12 колонок и постит его в Apps Script. Два предохранителя: пустая база не выгружается (иначе таблица была бы затёрта начисто) и ответ скрипта проверяется на наличие "ok":true — страница авторизации Google иначе прошла бы как успех.

Импорт принимает 12 колонок или 11 (устаревшая выгрузка без колонки привязок). В устаревшем случае привязки восстанавливаются из текущей базы, но только для предметов, у которых текст местоположения в таблице не трогали. Строки разной ширины и повторяющиеся id — отказ без записи в базу.

## 6.4. Устойчивость к сбоям Google

- CSV читается байтами и декодируется как UTF-8 вручную: без charset в Content-Type Spring взял бы ISO-8859-1 и молча испортил кириллицу.
- Таймауты: 10 секунд на соединение, 60 на чтение — иначе плановый импорт может повиснуть навсегда.
- Редиректы разрешены: Apps Script отвечает через цепочку перенаправлений.
- Любая ошибка возвращается как success:false, база при этом не трогается — скачивание идёт до deleteAll.

Это не теория: во время тестирования 18.09.2026 Apps Script вернул HTML-страницу Google Диска с 404 вместо CSV. Сервис ответил success:false, справочник остался нетронутым, следующая попытка

прошла штатно.

## 7. Развёртывание и эксплуатация

### 7.1. Прод

| Параметр            | Значение                                                                      |
|---------------------|-------------------------------------------------------------------------------|
| Приложение fly.io   | tkstorage-api, регион fra, домен tkstorage-api.fly.dev                        |
| Машина              | 1 GB памяти, 1 shared CPU, min_machines_running = 1                           |
| Автостоп            | auto_stop_machines = stop, auto_start_machines = true                         |
| Health check        | GET /health каждые 15 секунд, таймаут 10 секунд                               |
| Лимиты запросов     | soft 20, hard 25 одновременных                                                |
| Подключение к Mongo | секрет fly SPRING_DATA_MONGODB_URI (в репозитории лежит только локальный URI) |

```
fly deploy -a tkstorage-api      # сборка образа и раскатка
fly status -a tkstorage-api     # состояние машин
fly logs -a tkstorage-api       # живой поток логов
fly secrets list -a tkstorage-api # имена секретов (значения не показываются)
```

Логи fly не хранятся. Доступен только живой буфер — на практике это последние часы. Ночной прогон пишет одну строку с итогами, и если разбираться на следующий день, её уже нет. Ручной импорт не логируется вообще (в UserService нет ни одной строки логирования), его результат виден только вызывающему в HTTP-ответе.

### 7.2. Типовые операции

- Справочник устарел (в приложении не находится новый человек): POST /users/import/sheets. Занимает около минуты на 3000 анкет.
- После импорта проверьте GET /items/unbound — там окажутся предметы, чьи люди изменились в таблице.
- Разбор: предметы со свободным текстом привязываются к людям вручную через приложение; текст ФИО в карточке подсказывает, кого искать.
- Срочная выгрузка в таблицу: POST /items/export/sheets. Проверьте, что ответ success:true — иначе таблица не обновилась.
- Полная копия базы для анализа: GET /items/export/csv (или /items/export/xls).

## 8. Известные проблемы и технический долг

Раздел, с которого стоит начинать, если вы принимаете проект.

### 8.1. Хрупкий ключ: правка анкеты молча снимает привязки

Главная известная проблема. id человека — хэш ФИО и даты рождения, поэтому любая правка этих полей в таблице делает старый id несуществующим, и сверка справочника снимает все привязки этого человека. Снятие происходит при ближайшем импорте, то есть до суток спустя после правки.

Подтверждённый случай (18.09.2026). Таблицу правили утром 17 сентября. Весь день система жила на старом справочнике, выдачи привязывались нормально. Ночной прогон 18 сентября в 04:30 подтянул правки: схлопнул 142 дубля и снял 17 привязок у трёх человек. Утром эти предметы обнаружились на экране разбора без каких-либо следов в истории; причину удалось установить только по строке в логах fly, которая через несколько часов была бы уже недоступна.

Предлагаемое решение — ремап вместо снятия: перед перезаписью справочника снять срез id -> (ФИО, сырая дата), после импорта для исчезнувших id пересчитать ключ новой нормализацией и переписать locationUserId у предметов. Это закрывает и правки в таблице, и будущие изменения самой нормализации. Не реализовано.

## 8.2. Наблюдаемость

- Снятие привязки не попадает в history — в карточке предмета не видно, что и почему произошло.
- Ручной импорт не логируется; ночной пишет одну строку в эфемерный лог fly.
- Нет постоянного журнала синхронизаций. Предлагалось писать результат каждого прогона в отдельную коллекцию (время, источник, счётчики, список снятых привязок).

## 8.3. Безопасность

- Аутентификации нет. Единственная проверка — заголовок версии приложения, который подставляется тривиально.
- GET /items/export/csv отдаёт всю базу снаряжения вообще без заголовка версии.
- Поиск людей отдаёт ФИО, дату рождения и телеграм любому, кто знает адрес сервиса.
- CORS разрешён для всех origin.
- В application.properties лежат боевые URL Apps Script, то есть репозиторий содержит рабочие точки доступа к таблицам клуба.

## 8.4. Данные и совместимость

- В справочнике остаются полные тёзки с разными датами рождения — различить их в поиске можно только по дате и телеграму. Часть таких пар — действительно разные люди, часть — незамеченные дубли.
- Все значения хранятся строками: вес, даты, номера. Сортировка предметов держится на приведении id к числу.
- Зона GMT+3 захардкожена в двух сервисах.
- Устаревший 11-колоночный импорт снаряжения сохраняет привязки эвристикой по тексту.
- В фильтре версий исключён путь /items/export, которого в контроллере нет — мёртвое правило, оставшееся от прежней схемы выгрузки.
- Перезапись справочника (deleteAll + saveAll) не транзакционна: есть короткое окно, в котором справочник пуст.

## 8.5. Тесты

Автотестов, кроме юнит-тестов нормализации дат, нет. Проверка изменений сегодня выглядит так: поднять локальный инстанс с отключённым кроном, прогнать импорт справочника на реальной выгрузке и сравнить счётчики и идентификаторы до и после. Этого достаточно, чтобы поймать смену ключей, но не покрывает логику привязок и историю.

## 9. Хронология значимых изменений

| Дата             | Что произошло                                                                     |
|------------------|-----------------------------------------------------------------------------------|
| 13.05.2026       | Первая версия API, перенос логики из Apps Script в backend                        |
| 17.06–19.06.2026 | Выгрузка и импорт гугл-таблиц, csv/xls, история и поиск                           |
| 10.09.2026       | Первая версия справочника людей                                                   |
| 16.09.2026       | Отключено автоматическое сопоставление предметов с людьми по тексту               |
| 17.09.2026       | Починен планировщик, добавлена автовыгрузка в Apps Script; справочник влит в main |
| 18.09.2026       | Поддержка ISO-дат в нормализации (коммит "add new data format")                   |

Про последнюю строку стоит знать подробности: интеграция Яндекс.Формы с таблицей анкет, подключённая неделей раньше, начала писать даты рождения через дефис. Все анкеты, поданные после этого, получили ключ, отличный от ключа тех же людей в старом формате. Фикс привёл оба написания к одному ключу; на боевой базе это схлопнуло 20 пар дублей (162 против 142) и сменило id у людей, чьи анкеты были только в новом формате — их привязки при ближайшем импорте отправились в разбор.

## 10. Глоссарий

| Термин             | Значение                                                                            |
|--------------------|-------------------------------------------------------------------------------------|
| Привязка           | Связь предмета с записью справочника через locationUserId                           |
| Разбор             | Экран приложения с предметами, у которых есть текст местоположения, но нет привязки |
| Справочник         | Коллекция club_users, полностью перезаписываемая из гугл-таблицы анкет              |
| Бизнес-ключ        | Поле id, вычисляемое из содержимого записи, в отличие от mongold                    |
| Служебные места    | Записи "Склад" и "Неизвестно" с фиксированными id warehouse и unknown               |
| Протухшая привязка | Ссылка на id, которого больше нет в справочнике                                     |
| Apps Script        | Скрипты Google, через которые сервис читает и пишет гугл-таблицы                    |